

# NVIDIA-Certified Associate Generative AI LLMs Master Cheat Sheet

Organized quick-review glossary for exam recall, scenario mapping, and final revision.

**Prepared by: Yasir Alharbi**

Cybersecurity and AI enthusiast

LinkedIn: [www.linkedin.com/in/Yasir-T-Alharbi](https://www.linkedin.com/in/Yasir-T-Alharbi)

**How to use:** scan domains first, then use decision tree and scenario keywords for exam questions.

## Main Sections

- Exam Domains
- Repeated Exam Patterns
- AI Subject Terms Glossary
- Python Tools and Libraries
- Exam Decision Tree
- Common Exam Traps
- Formula and Metric Mini Sheet
- Scenario Keywords

Consolidated from the provided PDFs and domain screenshots. Definitions are intentionally short for fast exam review.

# Exam Domains

## Core ML & AI Fundamentals

**Weight:** 30%

### Concepts

**Artificial Intelligence (AI):** Systems that perform tasks requiring human intelligence.

**Machine Learning (ML):** Algorithms learn patterns from data.

**Deep Learning (DL):** Multi-layer neural networks learn representations automatically.

**Generative AI:** AI that creates text, images, code, audio, or data.

**Foundation Model:** Large pretrained model adaptable to many tasks.

**Large Language Model (LLM):** Transformer model trained on large text corpora.

**Supervised Learning:** Learns from labeled input-output examples.

**Unsupervised Learning:** Finds patterns in unlabeled data.

**Semi-supervised Learning:** Uses small labeled data plus large unlabeled data.

**Self-supervised Learning:** Creates labels from raw data itself.

**Reinforcement Learning:** Learns actions using rewards.

**Transfer Learning:** Reuses pretrained knowledge for a new task.

**Feature Extraction:** Freeze base model; train only final layers.

**Fine-tuning:** Continue training a pretrained model on task data.

**Feature Engineering:** Create useful model inputs from raw data.

**Representation Learning:** Model learns useful internal features.

**Embedding:** Dense vector representing meaning or features.

**Text Embedding:** Vector representation of text semantics.

**Word2Vec:** Learns word embeddings from text context.

**CBOW:** Predicts target word from surrounding context.

**Skip-gram:** Predicts context words from a target word.

**Overfitting:** Performs well on training data, poorly on unseen data.

**Underfitting:** Model too simple to learn patterns.

**Regularization:** Reduces overfitting.

**Dropout:** Randomly disables neurons during training.

**L1 Regularization:** Encourages sparse weights.

**L2 Regularization:** Penalizes large weights.

**Loss Function:** Measures prediction error.

**Gradient Descent:** Updates weights to reduce loss.

**Learning Rate:** Step size for weight updates.

**Backpropagation:** Computes gradients through network layers.

**Epoch:** One full pass through training data.

**Batch Size:** Samples processed per training step.

**Hyperparameter:** Config set before training.

**Parameter:** Learned model weight.

**Inference:** Running a trained model to produce output.

**Classification:** Predicts categories.

**Regression:** Predicts continuous values.

**Clustering:** Groups similar unlabeled data.

**Anomaly Detection:** Finds unusual data points.

**Association Rule Mining:** Finds item co-occurrence rules.

**Time Series Forecasting:** Predicts future ordered values.

**Data Augmentation:** Creates varied training examples.

**Normalization:** Scales values for stable training.

**Standardization:** Centers/scales features by mean and variance.

**Padding:** Adds tokens/pixels to standardize size.

**Masking:** Prevents attention to irrelevant positions.

## Architectures

**Dense Network:** Fully connected neural network.

**CNN:** Learns spatial features from images.

**Convolution:** Sliding filter extracts local patterns.

**Kernel/Filter:** Small matrix detecting image features.

**Stride:** Step size of convolution movement.

**Pooling:** Reduces spatial size.

**Max Pooling:** Keeps strongest local activation.

**Average Pooling:** Averages local activations.

**RNN:** Processes sequential data step by step.

**LSTM:** RNN variant for long dependencies.

**GRU:** Simpler gated RNN variant.

**Transformer:** Attention-based architecture for sequences.

**Encoder-only Model:** Reads input for understanding tasks.

**Decoder-only Model:** Generates text autoregressively.

**Encoder-decoder Model:** Converts input sequence to output sequence.

**BERT:** Bidirectional encoder model.

**GPT:** Decoder-only autoregressive model.

**T5:** Encoder-decoder text-to-text model.

|                                                                    |
|--------------------------------------------------------------------|
| <b>BART:</b> Encoder-decoder generation model.                     |
| <b>Vision Transformer (ViT):</b> Treats image patches like tokens. |
| <b>Autoencoder:</b> Compresses and reconstructs data.              |
| <b>VAE:</b> Generative autoencoder with latent distribution.       |
| <b>GAN:</b> Generator and discriminator compete.                   |
| <b>Diffusion Model:</b> Learns to denoise step by step.            |
| <b>CLIP:</b> Aligns image and text embeddings.                     |
| <b>GNN:</b> Learns from graph nodes and edges.                     |
| <b>Multimodal LLM:</b> Processes multiple data types together.     |

## Transformer Concepts

|                                                                                            |
|--------------------------------------------------------------------------------------------|
| <b>Self-attention:</b> Tokens attend to other tokens in same sequence.                     |
| <b>Cross-attention:</b> Decoder attends to encoder outputs.                                |
| <b>Multi-head Attention:</b> Multiple attention views in parallel.                         |
| <b>Query (Q):</b> Vector used to search attention targets.                                 |
| <b>Key (K):</b> Vector used for attention matching.                                        |
| <b>Value (V):</b> Vector containing retrieved information.                                 |
| <b>Scaled Dot-product Attention:</b> Attention using QK scores scaled by $\sqrt{d_k}$ .    |
| <b><math>\sqrt{d_k}</math> Scaling:</b> Prevents large dot products destabilizing softmax. |
| <b>Positional Encoding:</b> Adds token order information.                                  |
| <b>Sinusoidal Encoding:</b> Original fixed transformer position signal.                    |
| <b>RoPE:</b> Rotary position embedding for long contexts.                                  |
| <b>Causal Mask:</b> Prevents decoder from seeing future tokens.                            |
| <b>Padding Mask:</b> Ignores padded tokens.                                                |
| <b>Feed-forward Network (FFN):</b> Per-token nonlinear transformation.                     |
| <b>Residual Connection:</b> Adds input back to layer output.                               |
| <b>LayerNorm:</b> Normalizes activations inside transformer layers.                        |
| <b>RMSNorm:</b> Efficient LayerNorm alternative.                                           |
| <b>Context Window:</b> Maximum tokens a model can process.                                 |
| <b>KV Cache:</b> Stores attention keys/values for faster decoding.                         |
| <b>Grouped-query Attention:</b> Shares KV heads to reduce inference memory.                |
| <b>Sliding Window Attention:</b> Attends to local token windows.                           |
| <b>FlashAttention:</b> Faster memory-efficient attention.                                  |
| <b>Mixture of Experts (MoE):</b> Activates only selected expert submodels.                 |

## Applications

|                                                           |
|-----------------------------------------------------------|
| <b>RAG Systems:</b> Retrieve knowledge before generation. |
|-----------------------------------------------------------|

|                                                               |
|---------------------------------------------------------------|
| <b>Chatbots:</b> Conversational LLM applications.             |
| <b>Summarizers:</b> Shorten text while preserving meaning.    |
| <b>Translation:</b> Convert text between languages.           |
| <b>Sentiment Analysis:</b> Classify emotional tone.           |
| <b>NER:</b> Detect named entities in text.                    |
| <b>Token Classification:</b> Label each token.                |
| <b>Text Classification:</b> Label whole text.                 |
| <b>Question Answering:</b> Answer from context or generation. |
| <b>Extractive QA:</b> Selects answer span from passage.       |
| <b>Generative QA:</b> Produces free-form answer.              |
| <b>Image Classification:</b> Predicts image class.            |
| <b>Object Detection:</b> Finds objects in images.             |
| <b>Speech Recognition:</b> Converts audio to text.            |
| <b>Fraud Detection:</b> Flags suspicious behavior.            |
| <b>Customer Churn Prediction:</b> Predicts customer loss.     |
| <b>Recommendation Systems:</b> Suggest items/users/content.   |
| <b>Drug Discovery:</b> Uses models for molecules/proteins.    |

## Frameworks, Libraries, Platforms, and Tools Map

|                                                                         |
|-------------------------------------------------------------------------|
| <b>PyTorch:</b> Flexible deep learning framework.                       |
| <b>TensorFlow:</b> Production deep learning framework.                  |
| <b>Keras:</b> High-level TensorFlow model API.                          |
| <b>JAX:</b> High-performance autodiff/array framework.                  |
| <b>scikit-learn:</b> Classical ML and metrics toolkit.                  |
| <b>Hugging Face Transformers:</b> Pretrained transformer model library. |
| <b>Hugging Face Datasets:</b> Dataset loading/preprocessing library.    |
| <b>Hugging Face PEFT:</b> Parameter-efficient fine-tuning tools.        |
| <b>Hugging Face TRL:</b> RLHF/DPO training utilities.                   |
| <b>spaCy:</b> Production NLP pipeline library.                          |
| <b>NLTK:</b> Traditional NLP toolkit.                                   |
| <b>SentenceTransformers:</b> Embedding models for semantic search.      |
| <b>LangChain:</b> Framework for LLM apps and chains.                    |
| <b>LangGraph:</b> Stateful graph workflows for agents.                  |
| <b>LlamaIndex:</b> Data connectors and RAG framework.                   |
| <b>NeMo Toolkit:</b> NVIDIA LLM customization framework.                |
| <b>NeMo Curator:</b> Dataset curation and filtering pipeline.           |
| <b>NeMo Guardrails:</b> Safety/rail framework for LLM apps.             |

|                                                                    |
|--------------------------------------------------------------------|
| <b>NVIDIA NIM:</b> Optimized inference microservice containers.    |
| <b>TensorRT-LLM:</b> Optimized LLM inference library.              |
| <b>Triton Inference Server:</b> Production model serving platform. |
| <b>RAPIDS:</b> GPU data science Python ecosystem.                  |
| <b>cuDF:</b> GPU DataFrame library.                                |
| <b>cuML:</b> GPU machine learning algorithms.                      |
| <b>cuGraph:</b> GPU graph analytics.                               |
| <b>Dask-cuDF:</b> Distributed GPU DataFrames.                      |
| <b>NVIDIA DALI:</b> GPU data loading/preprocessing.                |
| <b>CuPy:</b> NumPy-like arrays on GPU.                             |
| <b>Numba:</b> JIT compiler for Python/GPU kernels.                 |
| <b>FAISS:</b> Local vector similarity search.                      |
| <b>Milvus:</b> Open-source vector database.                        |
| <b>Pinecone:</b> Managed vector database.                          |
| <b>Weaviate:</b> Semantic vector database.                         |
| <b>Chroma:</b> Lightweight local vector database.                  |
| <b>ONNX:</b> Open model exchange format.                           |
| <b>ONNX Runtime:</b> Runs ONNX models.                             |
| <b>Docker:</b> Container runtime.                                  |
| <b>Kubernetes:</b> Container orchestration system.                 |
| <b>Helm:</b> Kubernetes package manager.                           |
| <b>FastAPI:</b> Modern Python REST API framework.                  |
| <b>Flask:</b> Lightweight Python web framework.                    |
| <b>Streamlit:</b> Quick Python data apps.                          |
| <b>Gradio:</b> Quick ML demos/interfaces.                          |
| <b>MLflow:</b> Experiment and model tracking.                      |
| <b>Weights &amp; Biases:</b> Experiment tracking dashboard.        |
| <b>DVC:</b> Data/model version control.                            |
| <b>Ray:</b> Distributed Python computing.                          |
| <b>Ray Serve:</b> Scalable model serving.                          |
| <b>BentoML:</b> Package and serve ML models.                       |
| <b>Airflow:</b> Workflow orchestration.                            |
| <b>Prefect:</b> Python workflow orchestration.                     |
| <b>NumPy:</b> Python numeric computing package.                    |
| <b>Pandas:</b> CPU DataFrame analysis library.                     |
| <b>Polars:</b> Fast DataFrame processing library.                  |
| <b>SciPy:</b> Scientific computing and optimization.               |

**Matplotlib:** Basic Python plotting.

**Seaborn:** Statistical visualization library.

**Plotly:** Interactive visualization library.

**Jupyter Notebook:** Interactive notebooks.

**pytest:** Python testing framework.

**Great Expectations:** Data quality validation.

**Evidently:** ML drift and monitoring reports.

## Data Analysis Techniques

**Weight:** 14%

### Techniques

**Data Mining:** Extracts patterns from large data.

**Data Visualization:** Shows insights with charts.

**Statistical Metrics:** Quantify model/data behavior.

**Trend Analysis:** Finds patterns over time.

**Performance Comparison:** Compares models using metrics.

**Exploratory Data Analysis:** Understands distributions and relationships.

**Descriptive Statistics:** Summarizes data.

**Correlation Analysis:** Measures relationship between variables.

**Distribution Analysis:** Studies data shape/spread.

**Outlier Detection:** Finds abnormal values.

**Dimensionality Reduction:** Compresses features while preserving structure.

**PCA:** Linear dimensionality reduction.

**Nearest Neighbor Search:** Finds closest vectors/items.

**Cosine Similarity:** Compares vector orientation.

**Euclidean Distance:** Measures straight-line distance.

**Clustering Analysis:** Groups similar records.

**K-means:** Clusters around centroids.

**Hierarchical Clustering:** Builds nested cluster tree.

**Spectral Clustering:** Uses graph structure for clustering.

**Graph Analytics:** Analyzes nodes and edges.

**Knowledge Graph:** Structured entity-relation graph.

### Processes

**Data Collection:** Gather raw data.

**Data Cleaning:** Fix missing, duplicate, inconsistent data.

**Deduplication:** Removes repeated records.

|                                                                               |
|-------------------------------------------------------------------------------|
| <b>Filtering:</b> Removes low-quality or irrelevant data.                     |
| <b>Data Transformation:</b> Convert data into model-ready form.               |
| <b>Feature Scaling:</b> Normalize numeric ranges.                             |
| <b>Categorical Encoding:</b> Convert categories to numbers.                   |
| <b>Tokenization:</b> Break text into tokens.                                  |
| <b>Chunking:</b> Split documents for retrieval/context limits.                |
| <b>Data Labeling:</b> Assign ground-truth labels.                             |
| <b>Data Versioning:</b> Track dataset changes.                                |
| <b>Dataset Provenance:</b> Track data origin and usage rights.                |
| <b>Train/Validation/Test Split:</b> Separate training, tuning, final testing. |
| <b>Insight Generation:</b> Turn analysis into action.                         |

## Tools

|                                                             |
|-------------------------------------------------------------|
| <b>RAPIDS:</b> GPU-accelerated data science suite.          |
| <b>cuDF:</b> GPU DataFrame library; pandas-like.            |
| <b>cuML:</b> GPU ML algorithms.                             |
| <b>cuGraph:</b> GPU graph analytics.                        |
| <b>Dask-cuDF:</b> Distributed multi-GPU DataFrames.         |
| <b>NVTabular:</b> GPU feature engineering for tabular data. |
| <b>CUDA-X:</b> NVIDIA accelerated software libraries.       |
| <b>NVIDIA DALI:</b> GPU data loading/preprocessing.         |
| <b>TAO Toolkit:</b> Simplifies training and augmentation.   |
| <b>Pandas:</b> CPU data manipulation.                       |
| <b>Polars:</b> Fast DataFrame library.                      |
| <b>Dask:</b> Distributed Python data processing.            |

## Experimentation & Evaluation

**Weight:** 22%

### Methods

|                                                                |
|----------------------------------------------------------------|
| <b>A/B Testing:</b> Compare versions on real traffic.          |
| <b>Benchmarking:</b> Compare against standard baselines.       |
| <b>Cross-validation:</b> Evaluate across multiple data splits. |
| <b>K-fold Validation:</b> Split data into k train/test folds.  |
| <b>Hold-out Testing:</b> Use separate unseen test set.         |
| <b>Temporal Validation:</b> Test using future time data.       |
| <b>Statistical Testing:</b> Checks significance of results.    |
| <b>Human Evaluation:</b> Humans judge quality.                 |

**LLM-as-a-Judge:** LLM scores outputs.

**Error Analysis:** Systematically inspect failures.

**Ablation Study:** Remove components to measure impact.

**Stress Testing:** Test under heavy/extreme conditions.

**Adversarial Testing:** Test with hostile edge cases.

**Red Teaming:** Attack model to find risks.

**Canary Deployment:** Release to small user subset.

## Metrics

**Accuracy:** Fraction correct.

**Precision:** Correct positives among predicted positives.

**Recall:** Correct positives among actual positives.

**F1 Score:** Harmonic mean of precision and recall.

**Confusion Matrix:** Counts prediction categories.

**Specificity:** Correct negatives among actual negatives.

**AUC-ROC:** Ranking quality across thresholds.

**MAE:** Average absolute regression error.

**MSE:** Average squared regression error.

**RMSE:** Square root of MSE.

**R-squared:** Regression variance explained.

**Cross-entropy Loss:** Classification probability loss.

**BLEU:** N-gram overlap for translation.

**ROUGE:** Overlap metric for summaries.

**METEOR:** Translation quality metric.

**Perplexity:** Language model uncertainty.

**Groundedness:** Output supported by sources.

**Faithfulness:** Response matches retrieved evidence.

**Relevance:** Answer addresses the query.

**Coherence:** Output is logically organized.

**Toxicity Score:** Harmful language measurement.

**Latency:** Time per request.

**Throughput:** Requests/tokens per time.

**Error Rate:** Failed requests percentage.

**GPU Utilization:** How much GPU is used.

## RLHF

**RLHF:** Aligns LLM using human feedback.

**Human Feedback:** Human preference labels.

**Preference Pair:** Two outputs ranked by humans.

**Reward Model:** Predicts human preference score.

**PPO:** RL algorithm used in RLHF.

**DPO:** Directly optimizes preference data.

**Reward Hacking:** Model exploits reward incorrectly.

**Safety Alignment:** Makes outputs follow human values.

**Constitutional AI:** Uses written principles for alignment.

**SFT:** Supervised fine-tuning on instruction-response pairs.

## Software Development

**Weight:** 24%

### Development

**Model Deployment:** Serve models in production.

**API Endpoint:** Network interface for model calls.

**REST API:** HTTP-based service interface.

**gRPC:** High-performance service protocol.

**SDK:** Developer tools for using a platform.

**Containerization:** Package app and dependencies.

**Docker:** Container runtime.

**Kubernetes:** Container orchestration system.

**Helm Chart:** Kubernetes deployment package.

**Version Control:** Track code/model changes.

**Git:** Distributed version control tool.

**CI/CD:** Automated build, test, deployment.

**Unit Testing:** Tests small code units.

**Integration Testing:** Tests connected components.

**Model Validation Pipeline:** Automated model quality checks.

**Configuration File:** Stores settings/hyperparameters.

**YAML/JSON Config:** Common configuration formats.

**Logging:** Records events for debugging/audit.

**Asynchronous Programming:** Handles many I/O tasks concurrently.

**Scalability:** Handles growing workload.

**Horizontal Scaling:** Add more instances.

**Vertical Scaling:** Add more resources per instance.

**Caching:** Reuse frequent responses/results.

## Frameworks and App Tools

|                                                                     |
|---------------------------------------------------------------------|
| <b>LangChain:</b> Framework for LLM apps.                           |
| <b>LangGraph:</b> Orchestrates complex LLM workflows.               |
| <b>ChatNVIDIA:</b> LangChain connector for NVIDIA chat models.      |
| <b>NVIDIAEmbeddings:</b> LangChain connector for NVIDIA embeddings. |
| <b>Vector Database:</b> Stores/searches embeddings.                 |
| <b>FAISS:</b> Vector similarity search library.                     |
| <b>Milvus:</b> Vector database.                                     |
| <b>Pinecone:</b> Managed vector database.                           |
| <b>Weaviate:</b> Vector database with semantic search.              |
| <b>ONNX:</b> Open model exchange format.                            |
| <b>OpenAI-compatible API:</b> Standard chat/completion API shape.   |

## Monitoring

|                                                                  |
|------------------------------------------------------------------|
| <b>Performance Monitoring:</b> Tracks latency/throughput/errors. |
| <b>Model Monitoring:</b> Tracks quality/drift.                   |
| <b>Infrastructure Monitoring:</b> Tracks CPU/GPU/memory.         |
| <b>Drift Detection:</b> Finds distribution changes.              |
| <b>Quality Degradation Alert:</b> Warns when model worsens.      |
| <b>Automated Alerts:</b> Real-time issue notifications.          |
| <b>Audit Trail:</b> Record of actions and outputs.               |
| <b>User Feedback Loop:</b> Uses real usage feedback.             |

## Trustworthy AI Principles

**Weight:** 10%

## Ethics

|                                                            |
|------------------------------------------------------------|
| <b>Fairness:</b> Equitable treatment across groups.        |
| <b>Transparency:</b> Decisions are understandable.         |
| <b>Accountability:</b> Clear responsibility for outcomes.  |
| <b>Privacy:</b> Protect sensitive data.                    |
| <b>Consent:</b> Respect user data permission.              |
| <b>Beneficence:</b> System should benefit people.          |
| <b>Robustness:</b> Performs reliably under variation.      |
| <b>Explainability:</b> Explains model decisions.           |
| <b>Compliance:</b> Follows laws and regulations.           |
| <b>Model Card:</b> Documents model purpose, limits, risks. |
| <b>Data Sheet:</b> Documents dataset details.              |

## Bias

|                                                                    |
|--------------------------------------------------------------------|
| <b>AI Bias:</b> Unfair systematic model behavior.                  |
| <b>Data Bias:</b> Bias from training data.                         |
| <b>Sampling Bias:</b> Unrepresentative data collection.            |
| <b>Label Bias:</b> Biased annotation/ground truth.                 |
| <b>Algorithmic Bias:</b> Bias introduced by model/design.          |
| <b>Evaluation Bias:</b> Narrow tests hide failures.                |
| <b>Bias Detection:</b> Measure unfair group differences.           |
| <b>Bias Mitigation:</b> Reduce unfair outcomes.                    |
| <b>Fairness Constraints:</b> Enforce fairness during optimization. |
| <b>Adversarial Debiasing:</b> Train model to reduce bias signals.  |
| <b>Diverse Test Sets:</b> Evaluate across groups/scenarios.        |

## Safety and Guardrails

|                                                                      |
|----------------------------------------------------------------------|
| <b>NeMo Guardrails:</b> Controls safe LLM behavior.                  |
| <b>Colang:</b> Language for Guardrails dialogue flows.               |
| <b>Input Rail:</b> Filters user input before LLM.                    |
| <b>Output Rail:</b> Filters response after LLM.                      |
| <b>Topical Rail:</b> Keeps conversation on allowed topics.           |
| <b>Fact-checking Rail:</b> Verifies claims against sources.          |
| <b>Jailbreak:</b> Prompt that bypasses safety rules.                 |
| <b>Prompt Injection:</b> Malicious instruction inside input/context. |
| <b>Toxicity Detection:</b> Finds harmful language.                   |
| <b>PII Protection:</b> Protects personal information.                |
| <b>Differential Privacy:</b> Adds privacy-preserving noise.          |

## Key NVIDIA Technologies

### Platforms

|                                                                           |
|---------------------------------------------------------------------------|
| <b>NVIDIA NIM:</b> GPU-optimized inference microservice containers.       |
| <b>NVIDIA NeMo:</b> Framework for LLM training/fine-tuning/customization. |
| <b>TensorRT-LLM:</b> Optimizes LLM inference on NVIDIA GPUs.              |
| <b>Triton Inference Server:</b> Serves models with batching/versioning.   |
| <b>RAPIDS:</b> GPU-accelerated data science libraries.                    |
| <b>cuDF:</b> GPU DataFrame library.                                       |
| <b>cuML:</b> GPU ML algorithms.                                           |
| <b>cuGraph:</b> GPU graph analytics.                                      |
| <b>CUDA:</b> NVIDIA GPU programming platform.                             |

|                                                                       |
|-----------------------------------------------------------------------|
| <b>cuDNN:</b> GPU primitives for deep neural networks.                |
| <b>NCCL:</b> Multi-GPU communication library.                         |
| <b>NGC:</b> NVIDIA catalog for models, containers, SDKs.              |
| <b>BioNeMo:</b> Foundation models for biology/life sciences.          |
| <b>Megatron-LM:</b> Large-scale transformer training framework.       |
| <b>NeMo Curator:</b> Data curation/preprocessing pipeline.            |
| <b>NeMo Guardrails:</b> Safety framework for LLM apps.                |
| <b>NVIDIA AI Endpoints:</b> Hosted NVIDIA model APIs.                 |
| <b>DeepStream SDK:</b> Real-time video AI streaming.                  |
| <b>Omniverse:</b> 3D simulation/visualization platform.               |
| <b>Omniverse Nucleus:</b> Collaboration/storage for Omniverse assets. |

## NIM Details

|                                                                        |
|------------------------------------------------------------------------|
| <b>NIM Container:</b> Prebuilt optimized model server.                 |
| <b>NGC Distribution:</b> NIM images come from NGC.                     |
| <b>OpenAI API Compatibility:</b> Existing clients can switch easily.   |
| <b>REST/gRPC Endpoint:</b> NIM exposes production APIs.                |
| <b>TensorRT-LLM inside NIM:</b> Provides optimized inference.          |
| <b>Triton Backend:</b> Many NIMs use Triton internally.                |
| <b>Hardware Portability:</b> Same container across cloud/on-prem/edge. |

## NeMo Details

|                                                                   |
|-------------------------------------------------------------------|
| <b>SFT in NeMo:</b> Trains on prompt-response pairs.              |
| <b>RLHF in NeMo:</b> Reward model plus PPO alignment.             |
| <b>LoRA in NeMo:</b> Efficient adapter fine-tuning.               |
| <b>P-Tuning:</b> Learns soft prompt vectors.                      |
| <b>Prompt Tuning:</b> Trains prompt embeddings.                   |
| <b>Distributed Training:</b> Scales across GPUs/nodes.            |
| <b>Custom Tokenizer:</b> Domain-specific token splitting.         |
| <b>Megatron Integration:</b> Enables tensor/pipeline parallelism. |

## TensorRT-LLM Details

|                                                               |
|---------------------------------------------------------------|
| <b>In-flight Batching:</b> Adds requests into active batches. |
| <b>Continuous Batching:</b> Keeps GPU busy during serving.    |
| <b>Paged KV Cache:</b> Pages attention cache memory.          |
| <b>Custom CUDA Kernels:</b> Faster low-level GPU operations.  |
| <b>Quantization:</b> Lower precision for speed/memory.        |
| <b>INT8:</b> 8-bit inference precision.                       |

|                                                        |
|--------------------------------------------------------|
| <b>INT4:</b> 4-bit compression precision.              |
| <b>FP8:</b> Low precision floating-point.              |
| <b>AWQ:</b> Activation-aware weight quantization.      |
| <b>GPTQ:</b> Post-training quantization method.        |
| <b>Tensor Parallelism:</b> Splits tensors across GPUs. |

## Triton Details

|                                                                       |
|-----------------------------------------------------------------------|
| <b>Dynamic Batching:</b> Groups requests automatically.               |
| <b>Model Ensemble:</b> Chains models into one pipeline.               |
| <b>Model Versioning:</b> Hosts multiple versions.                     |
| <b>Concurrent Execution:</b> Runs multiple models together.           |
| <b>Multi-framework Serving:</b> Supports PyTorch, TF, ONNX, TensorRT. |
| <b>Model Repository:</b> Directory of deployable models/configs.      |
| <b>A/B Testing:</b> Compare model versions in production.             |

## NGC Details

|                                                                |
|----------------------------------------------------------------|
| <b>Model Catalog:</b> Find pretrained models.                  |
| <b>NIM Catalog:</b> Get production containers.                 |
| <b>Helm Charts:</b> Deploy on Kubernetes.                      |
| <b>Version Pinning:</b> Reproducible container/model versions. |
| <b>Model Signing:</b> Verifies container integrity.            |
| <b>Security Scanning:</b> Checks container vulnerabilities.    |

## Prompt Engineering Mastery

### Techniques

|                                                                          |
|--------------------------------------------------------------------------|
| <b>Prompt Engineering:</b> Craft prompts for desired outputs.            |
| <b>Zero-shot Prompting:</b> No examples provided.                        |
| <b>Few-shot Prompting:</b> Includes examples in prompt.                  |
| <b>One-shot Prompting:</b> Includes one example.                         |
| <b>Chain-of-Thought:</b> Encourages step-by-step reasoning.              |
| <b>Self-consistency:</b> Sample multiple reason paths; choose consensus. |
| <b>Tree of Thoughts:</b> Explore multiple reasoning branches.            |
| <b>Role Prompting:</b> Assign model a role/persona.                      |
| <b>System Prompt:</b> Sets top-level behavior/instructions.              |
| <b>Delimiters:</b> Separate instructions from data.                      |
| <b>Structured Output Prompting:</b> Request JSON/tables/schema.          |
| <b>Output Parsing:</b> Convert model output to usable format.            |

|                                                               |
|---------------------------------------------------------------|
| <b>Prompt Chaining:</b> Use outputs as later inputs.          |
| <b>Function Calling:</b> Let LLM call external tools.         |
| <b>Multi-turn Conversation:</b> Maintain dialogue context.    |
| <b>Domain Adaptation Prompting:</b> Tailor prompt to domain.  |
| <b>Safety Prompting:</b> Reduce harmful/off-policy responses. |

## Generation Controls

|                                                                 |
|-----------------------------------------------------------------|
| <b>Temperature:</b> Controls randomness.                        |
| <b>Low Temperature:</b> More deterministic output.              |
| <b>High Temperature:</b> More creative output.                  |
| <b>Top-k Sampling:</b> Sample from k most likely tokens.        |
| <b>Top-p Sampling:</b> Sample from probability nucleus.         |
| <b>Max Tokens:</b> Limits output length.                        |
| <b>Frequency Penalty:</b> Discourages repeated tokens.          |
| <b>Repetition Penalty:</b> Reduces repeated phrases.            |
| <b>Context Management:</b> Use context window efficiently.      |
| <b>Token Efficiency:</b> Minimize tokens while preserving task. |
| <b>Iterative Refinement:</b> Improve prompt based on output.    |

## Model Evaluation & Testing

### Evaluation

|                                                                 |
|-----------------------------------------------------------------|
| <b>Automated Metrics:</b> Quantitative model scores.            |
| <b>Human Evaluation:</b> Expert/user quality judgment.          |
| <b>Benchmark Dataset:</b> Standard comparison dataset.          |
| <b>GLUE:</b> NLP understanding benchmark.                       |
| <b>SuperGLUE:</b> Harder NLP benchmark.                         |
| <b>HELM:</b> Holistic LLM evaluation benchmark.                 |
| <b>MMLU:</b> Multi-domain knowledge benchmark.                  |
| <b>Domain-specific Test:</b> Custom test for target use case.   |
| <b>Adversarial Test:</b> Robustness against attacks/edge cases. |
| <b>Regression Test:</b> Ensures behavior does not break.        |
| <b>Golden Dataset:</b> Trusted reference test set.              |

### Validation

|                                                         |
|---------------------------------------------------------|
| <b>Cross-validation:</b> Reliable performance estimate. |
| <b>Hold-out Test:</b> Final unbiased test set.          |
| <b>Temporal Validation:</b> Future data validation.     |

|                                                     |
|-----------------------------------------------------|
| <b>Distribution Shift:</b> Train/test data differ.  |
| <b>Stress Test:</b> Performance under heavy load.   |
| <b>Calibration:</b> Confidence matches correctness. |
| <b>Generalization:</b> Works on unseen examples.    |

## Monitoring

|                                                              |
|--------------------------------------------------------------|
| <b>Drift Detection:</b> Input/output distribution changes.   |
| <b>Performance Tracking:</b> Continuous metric monitoring.   |
| <b>Error Analysis:</b> Investigate failure patterns.         |
| <b>User Feedback:</b> Real-world quality signal.             |
| <b>Automated Alert:</b> Notifies degradation.                |
| <b>Production Evaluation:</b> Measures live system behavior. |

# Repeated Exam Patterns

## Tool-to-Phase Mapping

|                                                                      |
|----------------------------------------------------------------------|
| <b>Data Prep:</b> RAPIDS, cuDF, DALI, NeMo Curator.                  |
| <b>Training:</b> NeMo, Megatron-LM, PyTorch, TensorFlow.             |
| <b>Fine-tuning:</b> NeMo, SFT, LoRA, P-Tuning, Prompt Tuning.        |
| <b>Alignment:</b> RLHF, reward model, PPO, DPO.                      |
| <b>Optimization:</b> TensorRT-LLM, quantization, batching, KV cache. |
| <b>Serving:</b> NIM, Triton, REST/gRPC.                              |
| <b>App Development:</b> LangChain, LangGraph, NVIDIA AI Endpoints.   |
| <b>Retrieval:</b> Embeddings, vector DB, FAISS, Milvus.              |
| <b>Safety:</b> NeMo Guardrails, Colang, rails.                       |
| <b>Catalog:</b> NGC.                                                 |
| <b>Monitoring:</b> Logs, drift, latency, throughput, alerts.         |

## RAG Pipeline

|                                                                 |
|-----------------------------------------------------------------|
| <b>User Query:</b> User asks a question.                        |
| <b>Query Embedding:</b> Convert query to vector.                |
| <b>Vector Database:</b> Stores searchable document vectors.     |
| <b>ANN Search:</b> Fast approximate nearest neighbor retrieval. |
| <b>Retriever:</b> Finds relevant chunks.                        |
| <b>Re-ranker:</b> Reorders retrieved chunks by relevance.       |
| <b>Context Injection:</b> Add chunks into prompt.               |
| <b>LLM Generation:</b> Produce grounded answer.                 |
| <b>Citation/Source Check:</b> Verify answer support.            |

**RAG Benefit:** Reduces hallucination and adds fresh knowledge.

**RAG Risk:** Bad retrieval causes bad answers.

## Fine-tuning Methods

**Full Fine-tuning:** Updates all model weights.

**PEFT:** Updates small trainable components.

**LoRA:** Low-rank trainable adapters.

**QLoRA:** Quantized base model plus LoRA.

**Adapter Tuning:** Small modules inserted in layers.

**Prompt Tuning:** Trains soft prompt tokens.

**P-Tuning:** Learns continuous prompt vectors.

**Instruction Tuning:** Teaches instruction following.

**Domain Adaptation:** Adapts model to specialized domain.

**Catastrophic Forgetting:** New training erases old skills.

**Gradient Checkpointing:** Saves memory by recomputing activations.

**Mixed Precision Training:** Uses FP16/BF16 for speed/memory.

## Optimization and Deployment

**Quantization:** Reduces precision for efficient inference.

**Pruning:** Removes less important weights.

**Distillation:** Smaller model learns from larger model.

**Batching:** Process requests together.

**Dynamic Batching:** Triton groups concurrent requests.

**Continuous/In-flight Batching:** Adds requests mid-generation.

**Speculative Decoding:** Smaller draft model speeds generation.

**KV Cache:** Speeds autoregressive decoding.

**Caching:** Stores common responses/results.

**Autoscaling:** Adjusts replicas to demand.

**Cold Start:** Delay when starting service/model.

**Edge Deployment:** Run model near device/user.

**Model Registry:** Tracks model versions/artifacts.

**Canary Release:** Small rollout before full release.

**Container Orchestration:** Manage containers at scale.

## Trust and Risk

**Hallucination:** Confident but false output.

**Grounding:** Connect output to evidence.

**Jailbreak:** Bypass safety guardrails.

**Prompt Injection:** Malicious prompt hidden in input.

**Data Leakage:** Sensitive data exposed.

**Copyright Risk:** Training data may violate rights.

**Privacy Risk:** User data mishandled.

**Bias Risk:** Unequal performance across groups.

**Toxic Output:** Harmful or offensive response.

**Robustness Risk:** Fails under unusual inputs.

**Accountability Gap:** Unclear responsibility for harm.

## Data Types and Preprocessing

**Image Data:** Resize, normalize, augment.

**Text Data:** Tokenize, pad, embed.

**Audio Data:** Spectrograms, normalization, augmentation.

**Tabular Data:** Scale numbers, encode categories.

**Time-series Data:** Normalize, window, forecast.

**Image Rotation:** Augments image angle.

**Image Flipping:** Mirrors images.

**Random Crop:** Varies image view.

**Color Jitter:** Changes brightness/contrast/color.

**Noise Injection:** Adds random noise for robustness.

**Synonym Replacement:** Text augmentation by synonyms.

**Back Translation:** Translate out and back to paraphrase.

**Random Insertion/Deletion:** Adds/removes words.

**Time Shift:** Audio time augmentation.

**Pitch Scaling:** Audio pitch augmentation.

## Common Confusions

**NIM vs Triton:** NIM is packaged model microservice; Triton is serving server.

**NIM vs NGC:** NIM runs models; NGC stores/distributes containers/models.

**NeMo vs NIM:** NeMo trains/customizes; NIM deploys/serves.

**TensorRT-LLM vs Triton:** TensorRT-LLM optimizes LLMs; Triton serves models.

**Guardrails vs RLHF:** Guardrails control runtime behavior; RLHF changes model alignment.

**RAPIDS vs Pandas:** RAPIDS/cuDF runs DataFrames on GPU; pandas runs on CPU.

**RAG vs Fine-tuning:** RAG retrieves knowledge; fine-tuning changes weights.

**LoRA vs Full Fine-tuning:** LoRA updates adapters; full updates all weights.

**BLEU vs ROUGE:** BLEU often translation; ROUGE often summarization.

**Precision vs Recall:** Precision avoids false positives; recall avoids false negatives.

**Latency vs Throughput:** Latency is delay; throughput is volume.

**Encoder vs Decoder:** Encoder understands; decoder generates.

**Self-attention vs Cross-attention:** Same sequence vs encoder-decoder attention.

# AI Subject Terms Glossary

## Core AI Terms

**Agent:** System that acts toward a goal.

**AI Pipeline:** Steps from data to deployed model.

**Algorithm:** Procedure for solving a task.

**Baseline:** Simple reference model/performance.

**Checkpoint:** Saved model state.

**Class Imbalance:** Unequal class distribution.

**Data Leakage:** Test information enters training.

**Decision Boundary:** Separates predicted classes.

**Embedding Space:** Vector space of learned meanings.

**Feature:** Input variable used by model.

**Generalization:** Performs well on unseen data.

**Ground Truth:** Correct reference answer/label.

**Heuristic:** Rule-of-thumb method.

**Label:** Correct output for supervised training.

**Latent Space:** Compressed hidden representation.

**Model Architecture:** Structure of model layers/components.

**Objective Function:** Function optimized during training.

**Pretraining:** Training on broad large data first.

**Prompt:** Input instructions sent to LLM.

**Representation:** Learned encoding of data.

**Sampling:** Choosing output tokens probabilistically.

**Training Corpus:** Dataset used for training.

**Validation Set:** Data used for tuning decisions.

## Neural Network Terms

**Activation Function:** Adds nonlinearity to networks.

**ReLU:** Outputs  $\max(0, x)$ .

**Sigmoid:** Maps values to 0-1.

**Tanh:** Maps values to -1 to 1.

**GELU:** Smooth activation common in transformers.

**Dead ReLU:** ReLU stuck outputting zero.

|                                                               |
|---------------------------------------------------------------|
| <b>Gradient:</b> Direction/size of parameter update.          |
| <b>Vanishing Gradient:</b> Gradients become too small.        |
| <b>Exploding Gradient:</b> Gradients become too large.        |
| <b>Optimizer:</b> Updates model weights.                      |
| <b>AdamW:</b> Adaptive optimizer with weight decay.           |
| <b>SGD:</b> Basic gradient descent optimizer.                 |
| <b>Weight Decay:</b> Regularization penalizing large weights. |
| <b>Batch Normalization:</b> Normalizes layer inputs.          |
| <b>Layer Normalization:</b> Normalizes features per sample.   |
| <b>Residual Block:</b> Layer with skip connection.            |
| <b>Parameter Count:</b> Number of learned weights.            |
| <b>Compute:</b> Processing needed for training/inference.     |
| <b>FLOPs:</b> Floating-point operations.                      |
| <b>VRAM:</b> GPU memory.                                      |

## NLP and LLM Terms

|                                                                        |
|------------------------------------------------------------------------|
| <b>NLP:</b> AI for human language.                                     |
| <b>Token:</b> Basic text unit processed by model.                      |
| <b>Tokenizer:</b> Converts text into tokens.                           |
| <b>Vocabulary:</b> Set of tokenizer tokens.                            |
| <b>Subword Tokenization:</b> Splits words into pieces.                 |
| <b>BPE:</b> Byte Pair Encoding tokenizer method.                       |
| <b>WordPiece:</b> Subword tokenizer used by BERT.                      |
| <b>SentencePiece:</b> Language-independent tokenizer.                  |
| <b>Padding Token:</b> Filler token for equal lengths.                  |
| <b>Attention Mask:</b> Marks real tokens vs padding.                   |
| <b>Autoregressive Model:</b> Predicts next token from previous tokens. |
| <b>Masked Language Modeling:</b> Predicts hidden tokens.               |
| <b>Next Sentence Prediction:</b> Predicts sentence continuity.         |
| <b>Causal Language Modeling:</b> Next-token prediction.                |
| <b>Bidirectional Attention:</b> Looks left and right.                  |
| <b>Unidirectional Attention:</b> Looks only previous tokens.           |
| <b>Instruction Following:</b> Model obeys user task instructions.      |
| <b>In-context Learning:</b> Learns from prompt examples.               |
| <b>Context Length:</b> Maximum tokens accepted.                        |
| <b>Stop Sequence:</b> Text that ends generation.                       |
| <b>Logit:</b> Raw score before probability.                            |

|                                                         |
|---------------------------------------------------------|
| <b>Softmax:</b> Converts scores to probabilities.       |
| <b>Decoding:</b> Converts model probabilities to text.  |
| <b>Greedy Decoding:</b> Always picks most likely token. |
| <b>Beam Search:</b> Keeps multiple candidate sequences. |
| <b>Nucleus Sampling:</b> Top-p sampling.                |

## Generative AI Terms

|                                                               |
|---------------------------------------------------------------|
| <b>Generative Model:</b> Creates new data.                    |
| <b>Discriminative Model:</b> Predicts labels/classes.         |
| <b>Generator:</b> GAN network that creates samples.           |
| <b>Discriminator:</b> GAN network that judges samples.        |
| <b>Forward Diffusion:</b> Adds noise to data.                 |
| <b>Reverse Diffusion:</b> Removes noise to generate.          |
| <b>Denoising:</b> Removing noise from data.                   |
| <b>Latent Diffusion:</b> Diffusion in compressed space.       |
| <b>Multimodal Model:</b> Handles multiple input types.        |
| <b>Image Captioning:</b> Generates text description of image. |
| <b>Text-to-Image:</b> Generates image from prompt.            |
| <b>Image-to-Text:</b> Generates text from image.              |
| <b>Speech-to-Text:</b> Converts audio to text.                |
| <b>Text-to-Speech:</b> Converts text to audio.                |
| <b>Synthetic Data:</b> Artificially generated training data.  |
| <b>Hallucination:</b> Plausible but incorrect generation.     |
| <b>Grounded Generation:</b> Generation supported by evidence. |

## Evaluation Terms

|                                                       |
|-------------------------------------------------------|
| <b>Benchmark:</b> Standard test for comparison.       |
| <b>Metric:</b> Numeric performance measure.           |
| <b>Loss:</b> Training error signal.                   |
| <b>Train Loss:</b> Error on training data.            |
| <b>Validation Loss:</b> Error on validation data.     |
| <b>Test Accuracy:</b> Final held-out accuracy.        |
| <b>False Positive:</b> Incorrect positive prediction. |
| <b>False Negative:</b> Incorrect negative prediction. |
| <b>True Positive:</b> Correct positive prediction.    |
| <b>True Negative:</b> Correct negative prediction.    |
| <b>ROC Curve:</b> TPR/FPR curve across thresholds.    |

**Confidence Score:** Model certainty estimate.

**Calibration Error:** Confidence mismatch.

**Inter-rater Agreement:** Human judge consistency.

**Regression Metric:** Score for numeric prediction.

**Classification Metric:** Score for class prediction.

**Retrieval Metric:** Score for search quality.

**Recall@k:** Relevant item appears in top k.

**MRR:** Mean reciprocal rank.

**NDCG:** Ranking quality metric.

## Data Science Terms

**Grid Search:** Tests all parameter combinations.

**Randomized Search:** Samples parameter combinations.

**Hyperparameter Tuning:** Finds best config.

**Cross-validation Fold:** One split in k-fold testing.

**ARIMA:** Time-series forecasting model.

**Autocorrelation:** Correlation with past values.

**Partial Autocorrelation:** Direct lag correlation.

**Stationarity:** Stable mean/variance over time.

**Seasonality:** Repeating time pattern.

**Trend:** Long-term direction.

**Intercept:** Baseline regression value.

**Coefficient:** Feature effect in regression.

**Residual:** Prediction error.

**Outlier:** Unusual data point.

**Missing Value:** Empty/unknown feature value.

**Imputation:** Fill missing values.

**One-hot Encoding:** Converts categories to binary columns.

## NVIDIA and Production Terms

**MIG:** Splits one GPU into isolated instances.

**Time-sharing:** Multiple workloads share GPU time.

**Model Signing:** Cryptographic verification of model/container.

**Vulnerability Scanning:** Checks security weaknesses.

**Version Pinning:** Locks exact model/container version.

**Helm:** Kubernetes package manager.

**Load Balancing:** Distributes traffic across servers.

|                                                                |
|----------------------------------------------------------------|
| <b>SLA:</b> Required service reliability/performance.          |
| <b>Observability:</b> Logs, metrics, traces for systems.       |
| <b>Telemetry:</b> Collected runtime measurements.              |
| <b>Throughput Bottleneck:</b> Component limiting system speed. |
| <b>CPU-GPU Transfer:</b> Data movement between CPU and GPU.    |
| <b>Zero-copy Interop:</b> Avoids unnecessary data copying.     |

## Python Tools and Libraries

### Core Python ML Stack

|                                                               |
|---------------------------------------------------------------|
| <b>Python:</b> Main language for AI/ML workflows.             |
| <b>NumPy:</b> Fast numerical arrays and math.                 |
| <b>Pandas:</b> CPU DataFrame data analysis.                   |
| <b>Polars:</b> Fast DataFrame processing library.             |
| <b>SciPy:</b> Scientific computing and optimization.          |
| <b>scikit-learn:</b> Classical ML and metrics toolkit.        |
| <b>Jupyter Notebook:</b> Interactive coding and exploration.  |
| <b>Matplotlib:</b> Basic Python plotting.                     |
| <b>Seaborn:</b> Statistical visualization on Matplotlib.      |
| <b>Plotly:</b> Interactive charts and dashboards.             |
| <b>Statsmodels:</b> Statistical models and time-series tools. |
| <b>NLTK:</b> Traditional NLP toolkit.                         |
| <b>spaCy:</b> Production NLP pipeline library.                |

### Deep Learning Libraries

|                                                          |
|----------------------------------------------------------|
| <b>PyTorch:</b> Flexible deep learning framework.        |
| <b>torch:</b> PyTorch tensor and neural network package. |
| <b>torch.nn:</b> PyTorch neural network layers.          |
| <b>torch.optim:</b> PyTorch optimizers.                  |
| <b>torchvision:</b> PyTorch image models/transforms.     |
| <b>torchaudio:</b> PyTorch audio processing.             |
| <b>TensorFlow:</b> Production deep learning framework.   |
| <b>Keras:</b> High-level TensorFlow model API.           |
| <b>TensorBoard:</b> Training visualization dashboard.    |
| <b>JAX:</b> High-performance array/autodiff framework.   |
| <b>Autograd:</b> Automatic gradient computation.         |
| <b>CUDA Tensor:</b> Tensor stored on NVIDIA GPU.         |

## Hugging Face Ecosystem

|                                                                     |
|---------------------------------------------------------------------|
| <b>Transformers:</b> Pretrained transformer models library.         |
| <b>AutoModel:</b> Loads model architecture automatically.           |
| <b>AutoTokenizer:</b> Loads matching tokenizer automatically.       |
| <b>Datasets:</b> Dataset loading and preprocessing library.         |
| <b>Tokenizers:</b> Fast tokenizer implementations.                  |
| <b>Accelerate:</b> Simplifies distributed/mixed-precision training. |
| <b>PEFT:</b> Parameter-efficient fine-tuning library.               |
| <b>TRL:</b> RLHF/DPO training utilities.                            |
| <b>Safetensors:</b> Safe fast model weight format.                  |
| <b>Model Hub:</b> Repository for pretrained models/datasets.        |
| <b>Pipeline API:</b> Quick task-based inference wrapper.            |

## LLM App Development

|                                                                    |
|--------------------------------------------------------------------|
| <b>LangChain:</b> Builds LLM apps and chains.                      |
| <b>LangGraph:</b> Stateful graph workflows for agents.             |
| <b>LlamaIndex:</b> Data connectors and RAG framework.              |
| <b>OpenAI Python SDK:</b> Calls OpenAI-compatible APIs.            |
| <b>NVIDIA AI Endpoints:</b> Hosted NVIDIA model APIs.              |
| <b>ChatNVIDIA:</b> LangChain chat connector for NVIDIA.            |
| <b>NVIDIAEmbeddings:</b> LangChain embedding connector.            |
| <b>FAISS Python:</b> Local vector similarity search.               |
| <b>Milvus SDK:</b> Connects Python to Milvus vector DB.            |
| <b>Pinecone SDK:</b> Connects Python to Pinecone.                  |
| <b>Weaviate Client:</b> Connects Python to Weaviate.               |
| <b>Chroma:</b> Lightweight local vector database.                  |
| <b>SentenceTransformers:</b> Embedding models for semantic search. |
| <b>tiktoken:</b> Token counting/tokenization utility.              |

## NVIDIA Python and GPU Tools

|                                                       |
|-------------------------------------------------------|
| <b>RAPIDS:</b> GPU data science Python ecosystem.     |
| <b>cuDF:</b> GPU DataFrame library.                   |
| <b>cudf.pandas:</b> GPU acceleration for pandas code. |
| <b>cuML:</b> GPU machine learning algorithms.         |
| <b>cuGraph:</b> GPU graph analytics.                  |
| <b>Dask-cuDF:</b> Distributed GPU DataFrames.         |
| <b>CuPy:</b> NumPy-like arrays on GPU.                |

|                                                                |
|----------------------------------------------------------------|
| <b>Numba:</b> JIT compiler for Python/GPU kernels.             |
| <b>NVIDIA DALI:</b> GPU data loading and preprocessing.        |
| <b>NVTabular:</b> GPU tabular feature engineering.             |
| <b>NeMo Toolkit:</b> Python framework for LLM customization.   |
| <b>NeMo Curator:</b> Dataset curation and filtering.           |
| <b>NeMo Guardrails:</b> Python safety/rail framework.          |
| <b>TensorRT Python API:</b> Build optimized inference engines. |
| <b>TensorRT-LLM:</b> Optimized LLM inference library.          |
| <b>Triton Client:</b> Python client for Triton inference.      |
| <b>NVIDIA NIM Client Pattern:</b> Call NIM via REST/gRPC.      |
| <b>NCCL Bindings:</b> Multi-GPU communication support.         |

## Data Processing Libraries

|                                                    |
|----------------------------------------------------|
| <b>PyArrow:</b> Columnar data and Parquet support. |
| <b>Parquet:</b> Efficient columnar storage format. |
| <b>JSONL:</b> One JSON object per line.            |
| <b>CSV:</b> Simple tabular text format.            |
| <b>BeautifulSoup:</b> HTML parsing.                |
| <b>Requests:</b> HTTP API calls.                   |
| <b>aiohttp:</b> Async HTTP client/server.          |
| <b>Regex:</b> Pattern matching for text cleaning.  |
| <b>Unidecode:</b> Text normalization utility.      |
| <b>pydantic:</b> Data validation and schemas.      |

## Serving and API Libraries

|                                                           |
|-----------------------------------------------------------|
| <b>FastAPI:</b> Modern Python REST API framework.         |
| <b>Flask:</b> Lightweight Python web framework.           |
| <b>Uvicorn:</b> ASGI server for FastAPI.                  |
| <b>Gunicorn:</b> Production Python web server.            |
| <b>gRPC Python:</b> High-performance RPC services.        |
| <b>Streamlit:</b> Quick Python data apps.                 |
| <b>Gradio:</b> Quick ML demos/interfaces.                 |
| <b>ONNX Runtime:</b> Runs ONNX models.                    |
| <b>Triton Inference Server:</b> Production model serving. |
| <b>Docker SDK:</b> Control Docker from Python.            |
| <b>Kubernetes Client:</b> Manage Kubernetes from Python.  |

## Testing and Quality Tools

|                                                     |
|-----------------------------------------------------|
| <b>pytest:</b> Python testing framework.            |
| <b>unittest:</b> Built-in Python test framework.    |
| <b>mypy:</b> Static type checking.                  |
| <b>ruff:</b> Fast Python linter/formatter.          |
| <b>black:</b> Opinionated Python formatter.         |
| <b>pre-commit:</b> Runs checks before commits.      |
| <b>tox:</b> Tests across environments.              |
| <b>coverage.py:</b> Measures test coverage.         |
| <b>Great Expectations:</b> Data quality validation. |
| <b>Evidently:</b> ML drift and monitoring reports.  |

## Experiment Tracking and MLOps

|                                                             |
|-------------------------------------------------------------|
| <b>MLflow:</b> Tracks experiments and models.               |
| <b>Weights &amp; Biases:</b> Experiment tracking dashboard. |
| <b>DVC:</b> Data/model version control.                     |
| <b>Hydra:</b> Configuration management.                     |
| <b>OmegaConf:</b> Hierarchical config library.              |
| <b>Airflow:</b> Workflow orchestration.                     |
| <b>Prefect:</b> Python workflow orchestration.              |
| <b>Ray:</b> Distributed Python computing.                   |
| <b>Ray Serve:</b> Scalable model serving.                   |
| <b>BentoML:</b> Package and serve ML models.                |

## Python Concepts for Certification

|                                                            |
|------------------------------------------------------------|
| <b>Virtual Environment:</b> Isolates project dependencies. |
| <b>pip:</b> Python package installer.                      |
| <b>conda:</b> Environment and package manager.             |
| <b>requirements.txt:</b> Lists pip dependencies.           |
| <b>pyproject.toml:</b> Modern Python project config.       |
| <b>Asyncio:</b> Python async concurrency.                  |
| <b>Coroutine:</b> Async function execution unit.           |
| <b>Type Hint:</b> Optional Python type annotation.         |
| <b>Exception Handling:</b> Handles runtime errors.         |
| <b>Serialization:</b> Save/load data structures.           |
| <b>Pickle:</b> Python object serialization.                |
| <b>Environment Variable:</b> Runtime configuration value.  |
| <b>Logging Module:</b> Built-in structured logging.        |

# Exam Decision Tree

## NVIDIA Tool Choice

|                                                                       |
|-----------------------------------------------------------------------|
| <b>Need deploy model fast:</b> Choose NVIDIA NIM.                     |
| <b>Need OpenAI-compatible endpoint:</b> Choose NVIDIA NIM.            |
| <b>Need private/on-prem inference:</b> Choose NVIDIA NIM.             |
| <b>Need optimize LLM inference:</b> Choose TensorRT-LLM.              |
| <b>Need lower latency:</b> Choose TensorRT-LLM or batching.           |
| <b>Need higher throughput:</b> Choose TensorRT-LLM, Triton, batching. |
| <b>Need serve many frameworks:</b> Choose Triton.                     |
| <b>Need model versioning/A-B tests:</b> Choose Triton.                |
| <b>Need chain models server-side:</b> Choose Triton ensemble.         |
| <b>Need train or fine-tune LLM:</b> Choose NeMo.                      |
| <b>Need RLHF/SFT/LoRA:</b> Choose NeMo.                               |
| <b>Need curate training data:</b> Choose NeMo Curator.                |
| <b>Need GPU DataFrame processing:</b> Choose RAPIDS/cuDF.             |
| <b>Need GPU ML algorithms:</b> Choose cuML.                           |
| <b>Need GPU graph analytics:</b> Choose cuGraph.                      |
| <b>Need safety/policy controls:</b> Choose NeMo Guardrails.           |
| <b>Need block jailbreaks/injections:</b> Choose Guardrails rails.     |
| <b>Need find NVIDIA containers/models:</b> Choose NGC.                |
| <b>Need biology foundation models:</b> Choose BioNeMo.                |
| <b>Need video AI streaming:</b> Choose DeepStream.                    |
| <b>Need GPU preprocessing:</b> Choose DALI.                           |

## Model Method Choice

|                                                                       |
|-----------------------------------------------------------------------|
| <b>Need external/current knowledge:</b> Use RAG.                      |
| <b>Need domain behavior/style:</b> Use fine-tuning.                   |
| <b>Need cheap fine-tuning:</b> Use PEFT/LoRA.                         |
| <b>Need align to preferences:</b> Use RLHF/DPO.                       |
| <b>Need reduce model size:</b> Use distillation/pruning/quantization. |
| <b>Need faster decoding:</b> Use KV cache/speculative decoding.       |
| <b>Need reduce memory:</b> Use quantization/LoRA/checkpointing.       |
| <b>Need compare live versions:</b> Use A/B testing.                   |
| <b>Need unbiased final score:</b> Use held-out test set.              |
| <b>Need robust estimate:</b> Use cross-validation.                    |

# Common Exam Traps

**RAG vs Fine-tuning:** RAG adds knowledge; fine-tuning changes behavior/weights.

**NIM vs NGC:** NIM runs models; NGC stores models/containers.

**NIM vs Triton:** NIM is packaged service; Triton is serving infrastructure.

**NeMo vs NIM:** NeMo trains/customizes; NIM deploys.

**TensorRT-LLM vs Triton:** TensorRT optimizes; Triton serves.

**Guardrails vs RLHF:** Guardrails runtime controls; RLHF training alignment.

**cuDF vs Pandas:** cuDF GPU; pandas CPU.

**cuML vs scikit-learn:** cuML GPU ML; scikit-learn CPU ML.

**Latency vs Throughput:** Latency delay; throughput volume.

**Precision vs Recall:** Precision reduces false positives; recall reduces false negatives.

**F1 vs Accuracy:** F1 better for imbalance; accuracy can mislead.

**BLEU vs ROUGE:** BLEU translation; ROUGE summarization.

**Perplexity vs Accuracy:** Perplexity language uncertainty; accuracy exact correctness.

**Encoder-only vs Decoder-only:** Encoder understands; decoder generates.

**BERT vs GPT:** BERT bidirectional encoder; GPT autoregressive decoder.

**Self-attention vs Cross-attention:** Same sequence vs encoder-decoder link.

**Top-k vs Top-p:** Fixed count vs probability mass.

**Temperature 0 vs High:** Deterministic vs creative.

**Quantization vs Pruning:** Lower precision vs remove weights.

**Data Parallelism vs Tensor Parallelism:** Split data vs split model tensors.

**Pipeline Parallelism vs Tensor Parallelism:** Split layers vs split tensor operations.

**Validation Set vs Test Set:** Tune on validation; final score on test.

**Data Bias vs Algorithmic Bias:** Bad data vs biased model/design.

**Prompt Injection vs Jailbreak:** Hidden malicious instruction vs bypass attempt.

**Groundedness vs Relevance:** Supported by sources vs answers question.

# Formula and Metric Mini Sheet

## Classification

**Accuracy:** Correct predictions / all predictions.

**Precision:**  $TP / (TP + FP)$ .

**Recall:**  $TP / (TP + FN)$ .

**F1:**  $2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$ .

**False Positive:** Predicted positive, actually negative.

**False Negative:** Predicted negative, actually positive.

**Confusion Matrix:** Table of TP, FP, FN, TN.

**AUC-ROC:** Ranking quality across thresholds.

## Regression

**MAE:** Average absolute error.

**MSE:** Average squared error.

**RMSE:** Square root of MSE.

**R-squared:** Proportion of variance explained.

## NLP and LLM

**Perplexity:** Lower means better next-token prediction.

**Cross-entropy:** Penalizes wrong probability predictions.

**BLEU:** N-gram precision for translation.

**ROUGE:** N-gram recall for summarization.

**METEOR:** Translation metric with synonym/stem matching.

**Groundedness:** Claims supported by context.

**Faithfulness:** Output does not contradict source.

**Toxicity:** Harmful/offensive content score.

## Retrieval and RAG

**Cosine Similarity:** Measures vector angle similarity.

**Recall@k:** Relevant item appears in top k.

**Precision@k:** Relevant fraction in top k.

**MRR:** Average reciprocal rank of first relevant result.

**NDCG:** Ranking quality with position weighting.

## Production

**Latency:** Time for one request.

**Throughput:** Requests/tokens per second.

**Error Rate:** Failed requests / total requests.

**GPU Utilization:** Percent GPU busy.

**Memory Footprint:** RAM/VRAM required.

**Tokens/sec:** Generation speed.

## Scenario Keywords

### NVIDIA Tools

**"OpenAI-compatible API":** NIM.

**"Pre-packaged optimized container":** NIM.

|                                                   |
|---------------------------------------------------|
| "Run model locally/private cloud": NIM.           |
| "Pull from nvcr.io": NGC/NIM.                     |
| "Catalog of models/containers": NGC.              |
| "Helm chart": NGC/Kubernetes deployment.          |
| "Fine-tune LLM": NeMo.                            |
| "SFT/RLHF/LoRA/P-Tuning": NeMo.                   |
| "Reward model/PPO": RLHF with NeMo.               |
| "Data curation/filtering": NeMo Curator.          |
| "Block unsafe topics": NeMo Guardrails.           |
| "Colang": NeMo Guardrails.                        |
| "Input/output rails": NeMo Guardrails.            |
| "Optimize LLM latency": TensorRT-LLM.             |
| "Paged KV cache": TensorRT-LLM.                   |
| "INT4/INT8/FP8": TensorRT-LLM quantization.       |
| "Dynamic batching": Triton.                       |
| "Model ensemble": Triton.                         |
| "Model versioning": Triton.                       |
| "Serve PyTorch/TensorFlow/ONNX together": Triton. |
| "GPU pandas": cuDF.                               |
| "GPU data science": RAPIDS.                       |
| "GPU clustering/PCA/nearest neighbors": cuML.     |
| "GPU graph analytics": cuGraph.                   |

## Model and Data Methods

|                                                           |
|-----------------------------------------------------------|
| "Needs latest/private documents": RAG.                    |
| "Reduce hallucinations with sources": RAG.                |
| "Vector database": Embeddings/RAG.                        |
| "Semantic search": Embeddings/vector DB.                  |
| "Small GPU memory for tuning": LoRA/QLoRA.                |
| "Train only few parameters": PEFT.                        |
| "Human preferences": RLHF/DPO.                            |
| "Pairwise comparisons": Reward model/DPO.                 |
| "Reduce model size": Distillation/pruning/quantization.   |
| "Faster inference": Quantization, TensorRT-LLM, batching. |
| "Multiple GPUs too large model": Tensor parallelism.      |
| "Split layers across GPUs": Pipeline parallelism.         |
| "Same model copy on each GPU": Data parallelism.          |

## Evaluation and Safety

|                                                     |
|-----------------------------------------------------|
| "Imbalanced classes": F1/precision/recall.          |
| "False positives costly": Precision.                |
| "False negatives costly": Recall.                   |
| "Translation quality": BLEU/METEOR.                 |
| "Summarization quality": ROUGE/human eval.          |
| "Language model fluency": Perplexity.               |
| "Open-ended answers": Human eval/LLM-as-judge.      |
| "Production comparison": A/B testing.               |
| "Final unbiased score": Hold-out test set.          |
| "Distribution changes": Drift detection.            |
| "Harmful prompts": Red teaming/adversarial testing. |
| "Sensitive user data": Privacy/PII controls.        |
| "Demographic fairness": Bias/fairness evaluation.   |